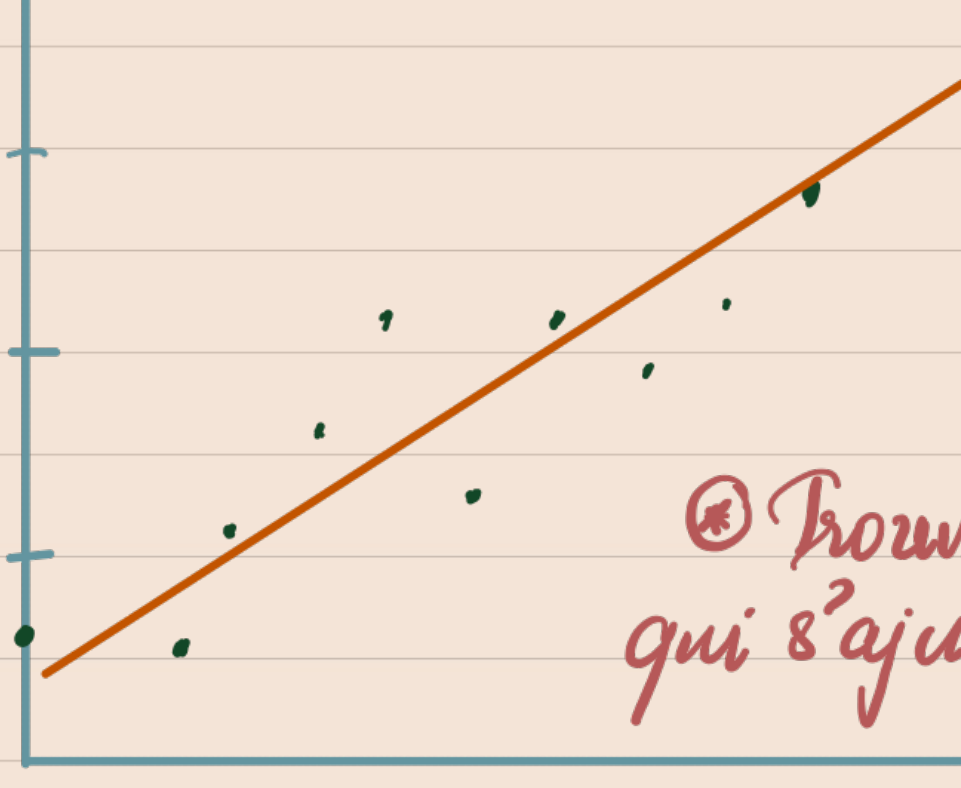


# Leçon 1: des Bases

## 1) Introduction MNIST

→ Def: Le machine learning est un domaine de l'IA qui consiste à programmer une machine pour que celle-ci apprenne à réaliser des tâches en étudiant des données de ces dernières

y



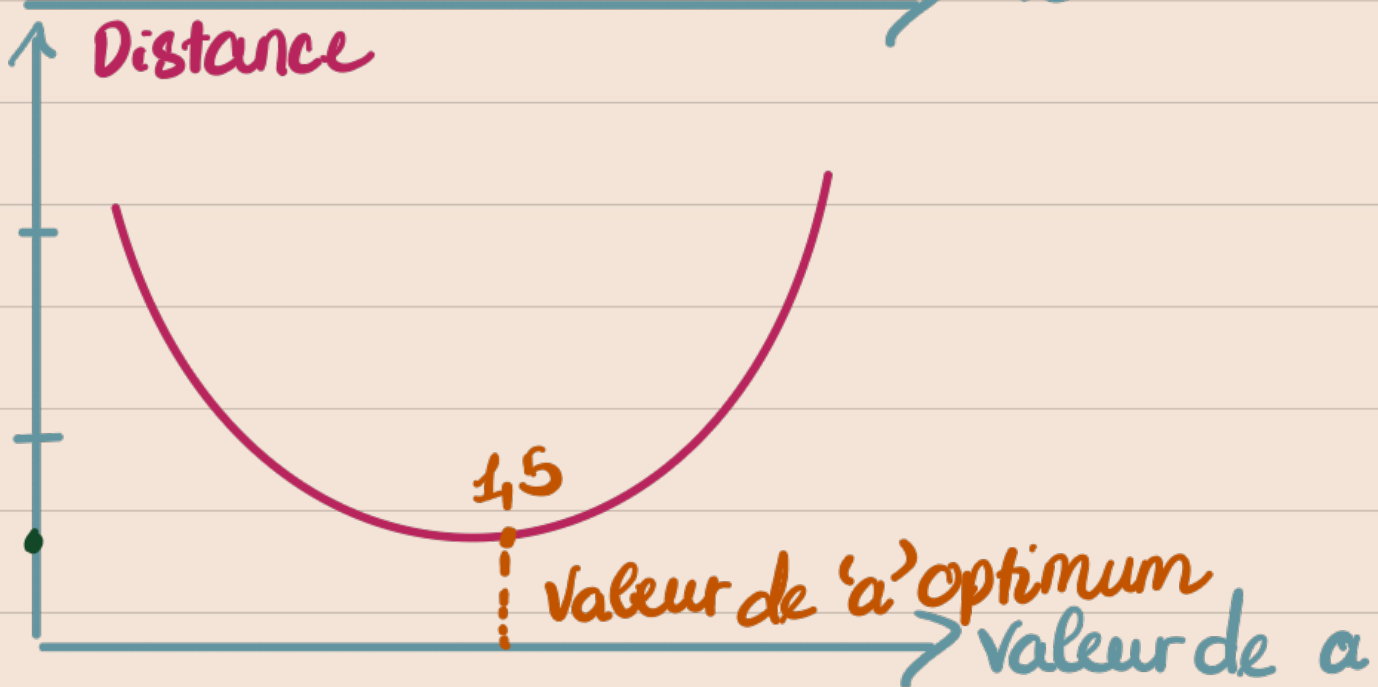
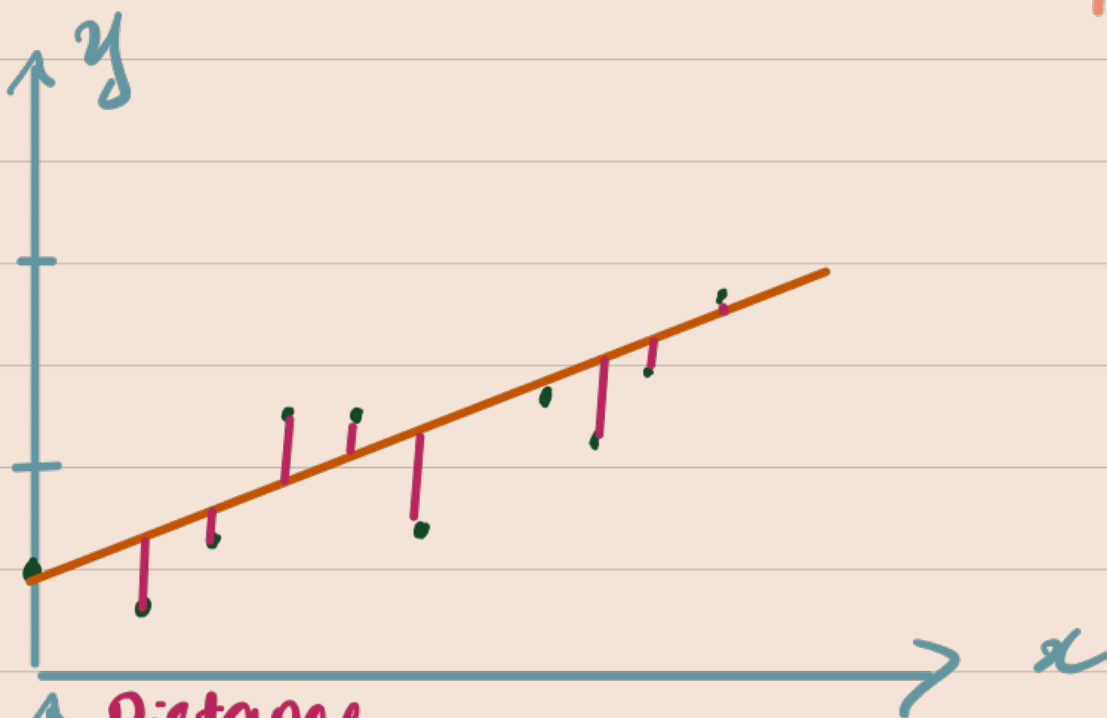
modèle  
 $f(x) = ax + b$

|             |
|-------------|
| <u>0,6</u>  |
| Valeur de a |
| <u>0,3</u>  |
| Valeur de b |

⊛ Trouver le modèle qui s'ajuste le mieux à nos données

## 2) des bases du Machine learning et du Deep learning

→ Pour cela, on programme dans la machine un **algorithme d'optimisation** qui va venir tester différentes valeurs de  $a$  &  $b$  jusqu'à obtenir la combinaison qui **minimise la distance** entre le modèle et les points.



## a) Le machine learning

→ Def: Développer un modèle, en se servant d'un algorithme d'optimisation pour minimiser les erreurs entre le modèle et nos données

### Machine learning

Modèles linéaires   Arbres de Décision  
Descente de Gradients   Algorithme CART

Support Vector Machines  
Marge Maximum

Note: les algos cités ici  
sont des exemples parmi  
d'autres

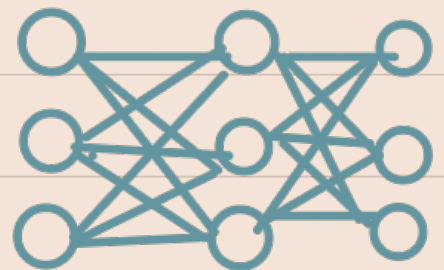
# Machine Learning

Modèles linéaires    Arbres de Décision  
Descente de Gradient    Algorithme CART

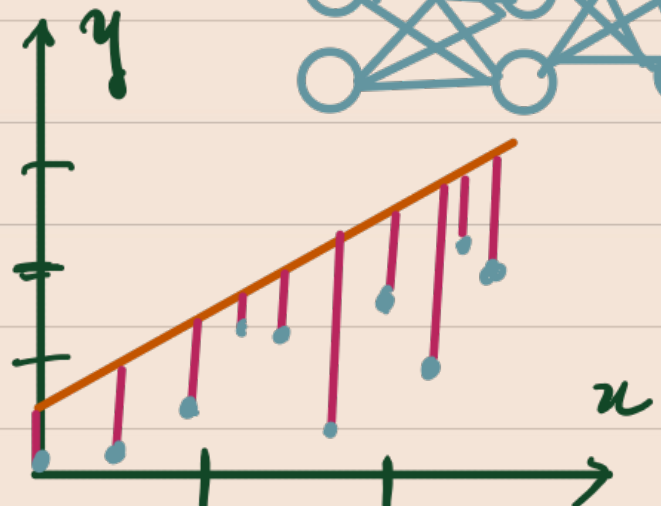
Support Vector Machines  
Marge Maximum

## Deep Learning

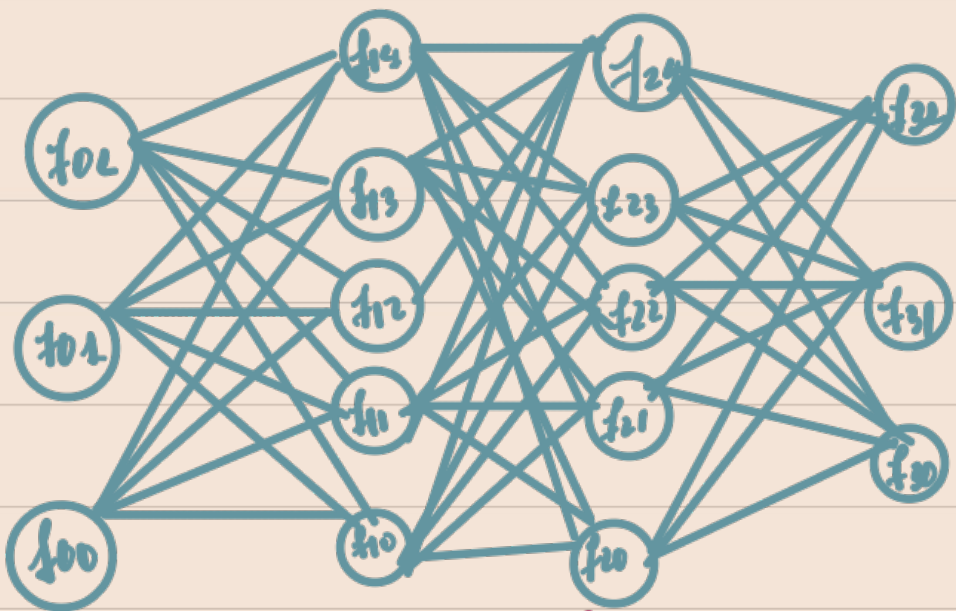
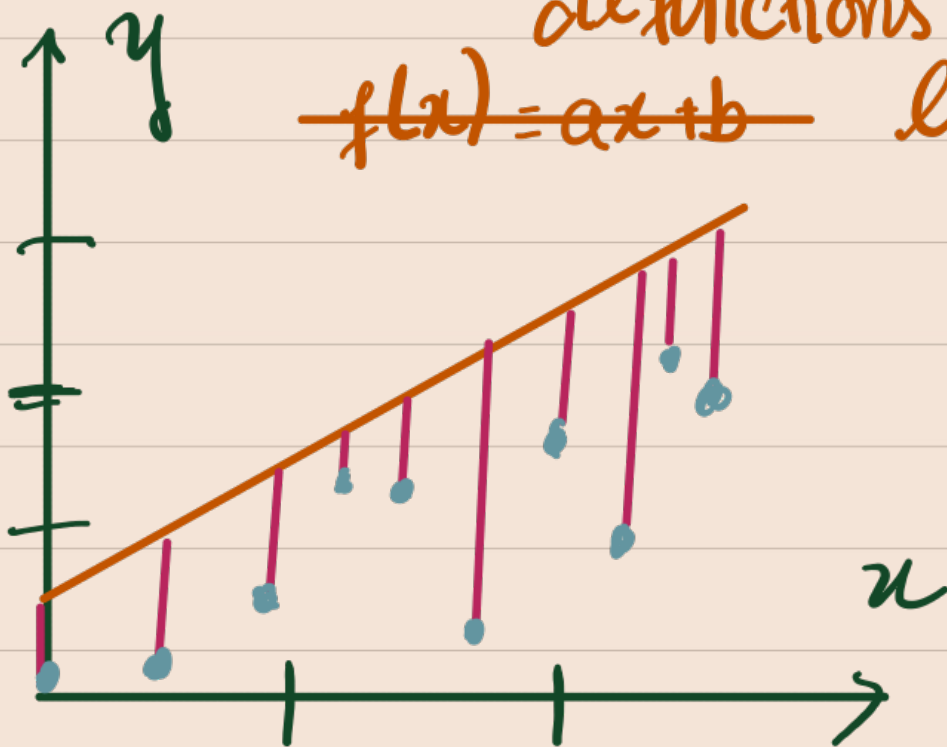
Réseaux de Neurons  
Artificiels



Données  
Modèle  
Algorithme  
d'Optimisation  
Minimisation  
des Erreurs



de modèle n'est pas une simple fonction  
(fonction linéaire) mais plutôt un réseau  
de fonctions connectées  
 ~~$f(x) = ax + b$~~  les unes aux autres

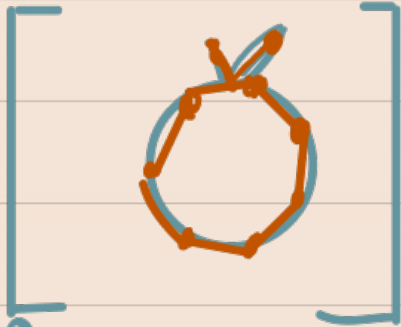


Réseau  
profonds  
(contenir  
de fonctions  
à l'intérieur)

↳ Capable d'apprendre à réaliser des  
tâches complexes.

# Tâches Complexes (Exemple):

Tout ce genre de choses



Reconnaître des  
objets



Identifier une personne  
sur une photo

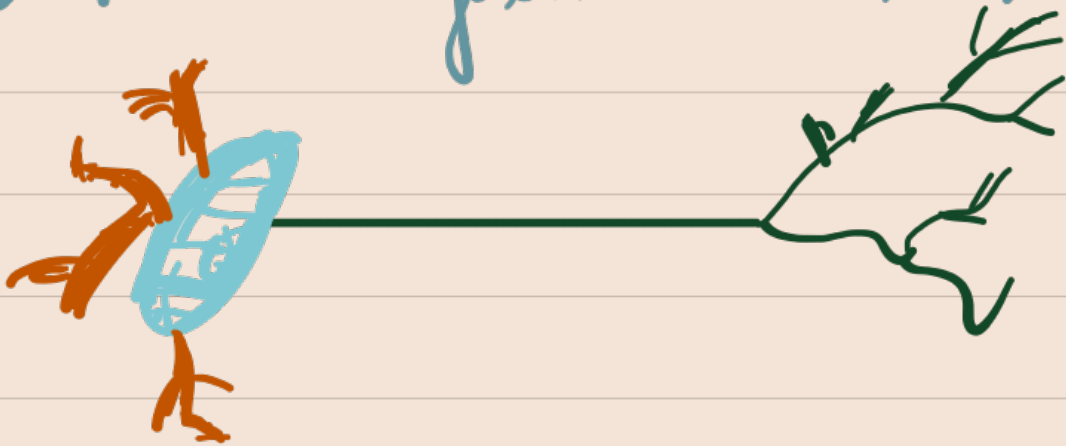


Conduire une voiture

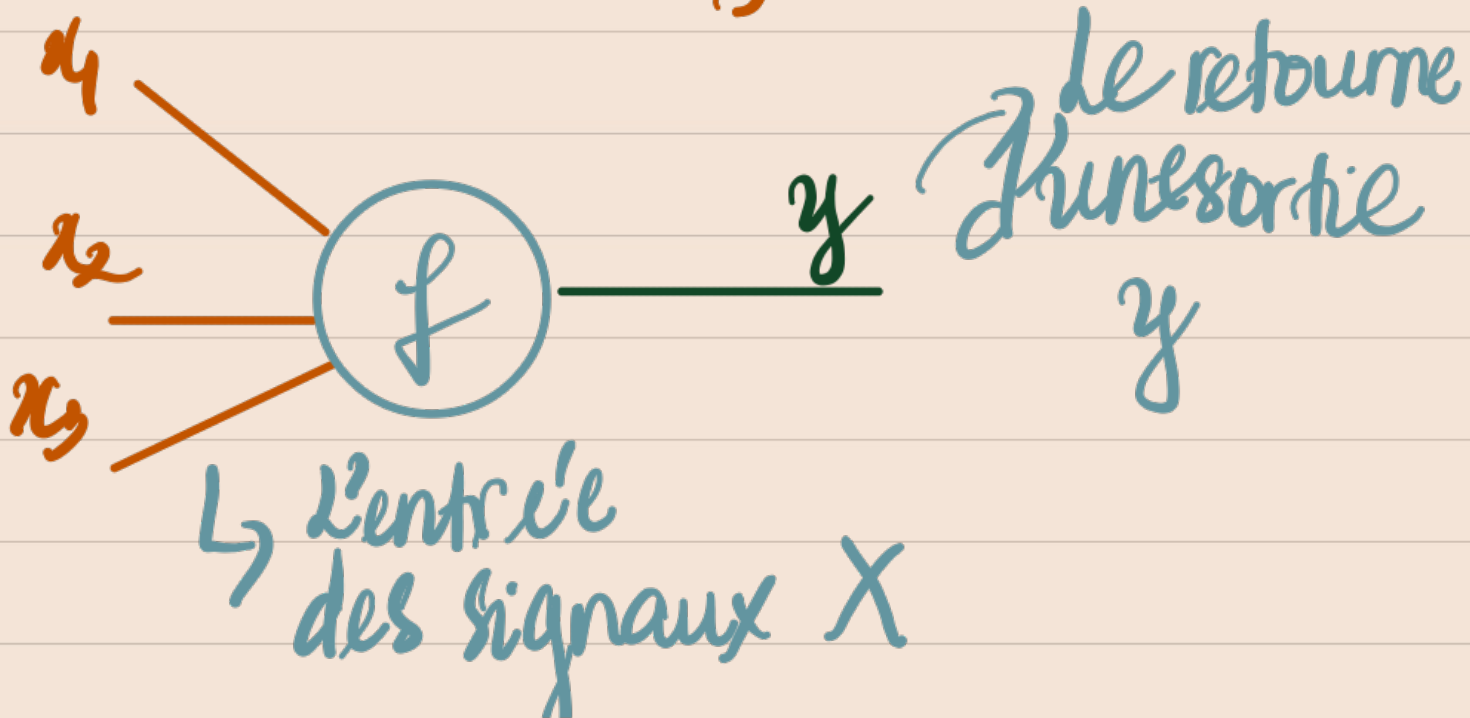


### 3) Le Neurone Artificiel de McCulloch et Pitts

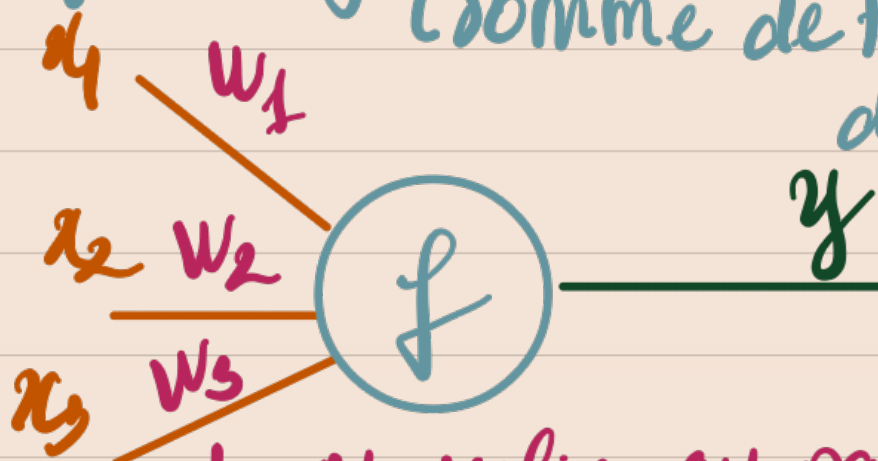
Les neurones sont des cellules excitables connectées les unes aux autres et ayant pour rôle de transmettre des infos dans notre système nerveux.



Ce que Warren McCulloch et Walter Pitts ont essayé de faire, c'est de **modéliser ce fonctionnement**.

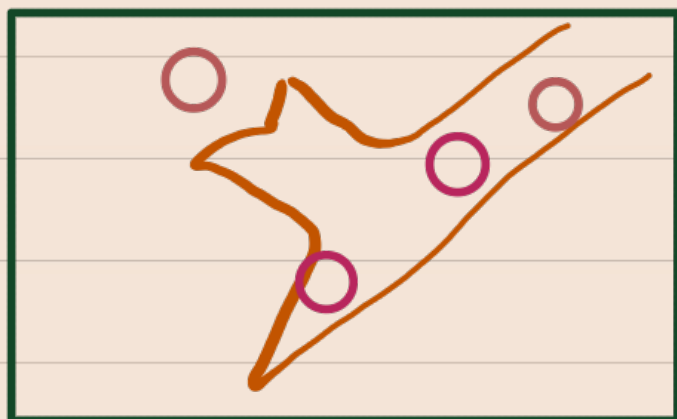


Agrégation  $f = w_1 x_1 + w_2 x_2 + w_3 x_3$   
 (Somme de tous les entrées du neurone)



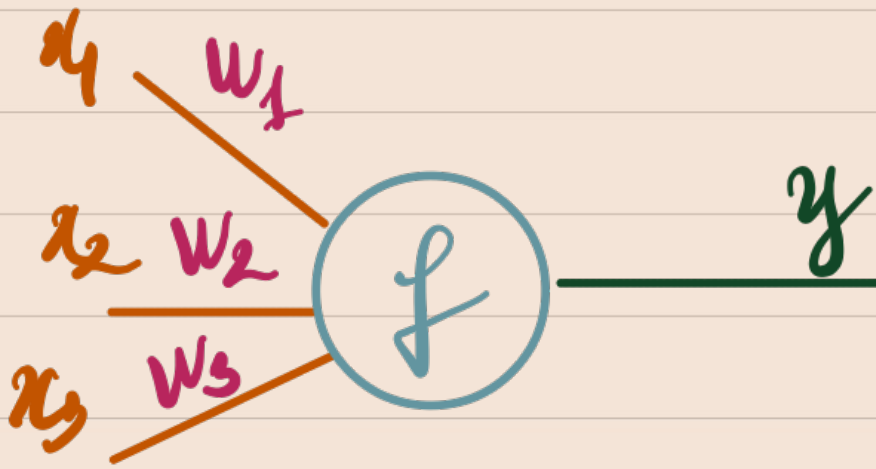
↳ Multiplier au passage  
 Chaque entrée par un coefficient  
 (fait l'activité synaptique)  
 = activation en IA!

Le fait que le signal soit excitateur  
 auquel cas  $w$  vaut  $+1$  ou bien  
 inhibiteur  $\therefore w$  vaut  $-1$



○: Excitateur  
 ○: Inhibiteur  
 (le neurone s'active)  
 de passe un certain  
 seuil

Activation  $\begin{cases} y = 1 \\ y = 0 \end{cases}$  si  $f \geq 0$   
 sinon



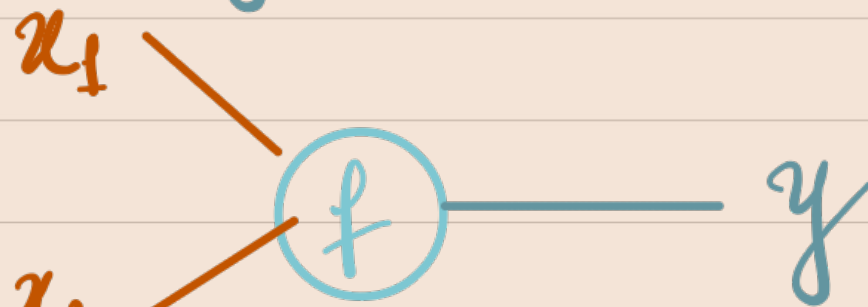
△ Une fois cette étape réalisée,  
on passe à la phase d'activation.  
On regarde le résultat du calcul  
effectué précédemment.

→ Threshold logic unit: le  
modèle n'était conçu que pour  
traiter des entrées logique qui  
valent soit 0 ou 1.

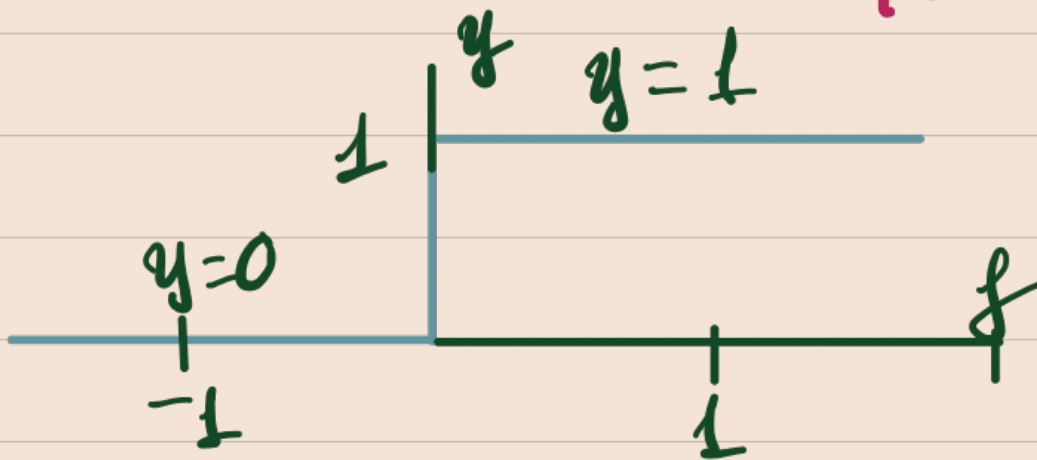
## 4) Le perceptron de Frank Rosenblatt

$$f = w_1 x_1 + w_2 x_2$$

$$\begin{cases} y = 1 & \text{si } f \geq 0 \\ y = 0 & \text{sinon} \end{cases}$$

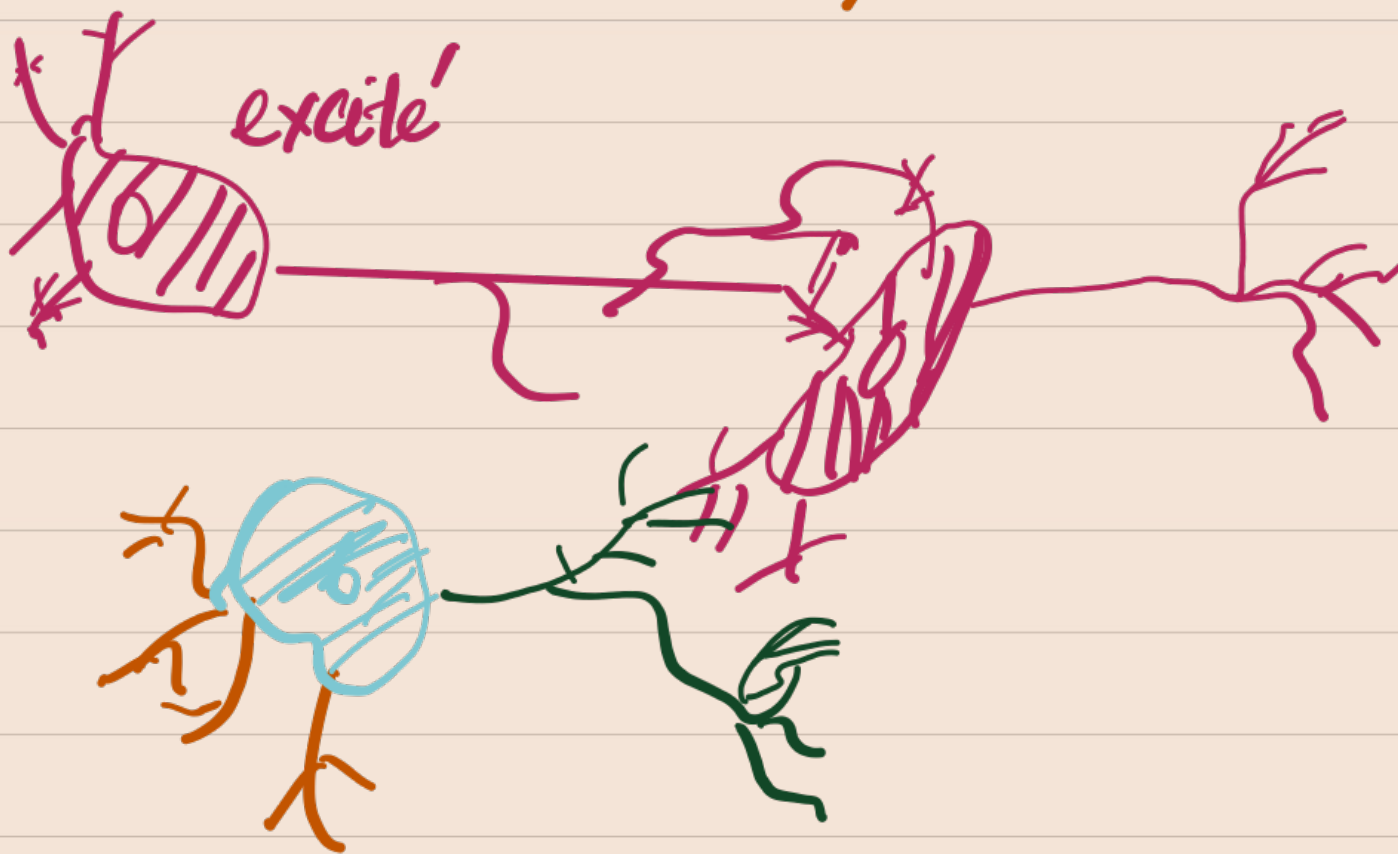


Algorithme d'apprentissage



→ Un neurone artificielle, qui s'active lorsque la somme pondérée de ses entrées dépasse un certain seuil, en général 0.

→ Théorie de Hebb: lorsque 2 neurones biologiques sont **excités conjointement** alors ils **renforcent** leur **lien synaptique**.



⇒ Ils renforcent les connexions qui les unissent. En neurosciences, c'est ce qu'on appelle la **plasticité synaptique**, et c'est ce qui permet à notre cerveau de construire sa mémoire, d'apprendre de nouvelles choses ou encore de faire

# de nouvelles associations

## Plasticité synaptique

Construire la mémoire Apprendre de nouvelles choses Faire des associations

de Perceptron (1957): Entraîner un neurone artificiel sur des données de références  $(X, y)$  pour que celui-ci renforce ses paramètres  $W$  à chaque fois qu'une entrée  $x$  est activée en même temps que la sortie  $y$  présente dans ces données.



renforcer ses paramètres  $w$  à chaque fois

qu'une entrée  $X$  est  
activée en même temps  
que la sortie  $y$  présente  
dans ces données

$$W = W + \alpha (y_{true} - y) X$$

$y$  : sortie produite par le neurone

$y_{true}$  : sortie de référence

$X$  : entrée du neurone

$\alpha$  : vitesse d'apprentissage



$$W = W + \alpha (1 - 0) X$$

$$= W + \alpha X$$

↳ augmenter d'un petit alpha  
( $x_1 = 1$ )  
( $x_0 = 0$ )

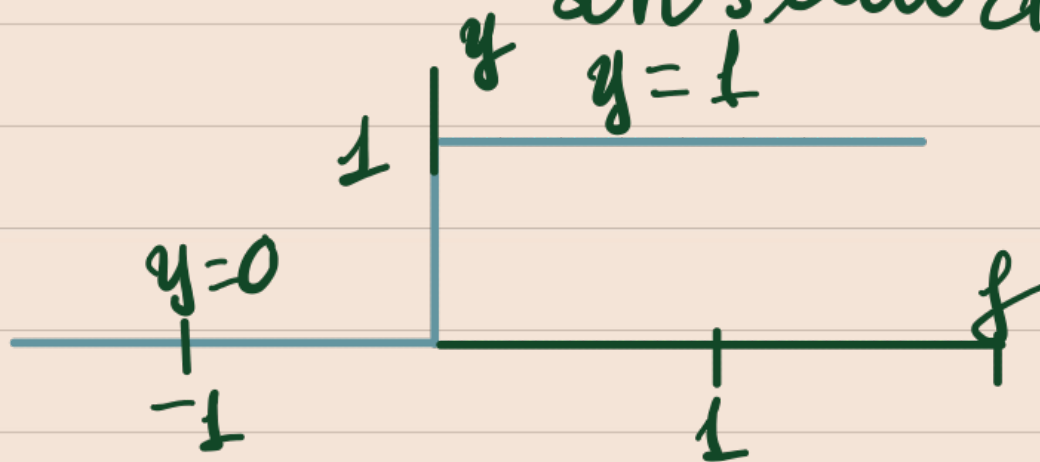
renforcé ↙

$$w_1 = w_1 + \alpha$$

$$w_2 = w_2$$

$$f = w_1 x_1 + w_2 x_2$$

Rapprochera le neurone de son seuil d'activation



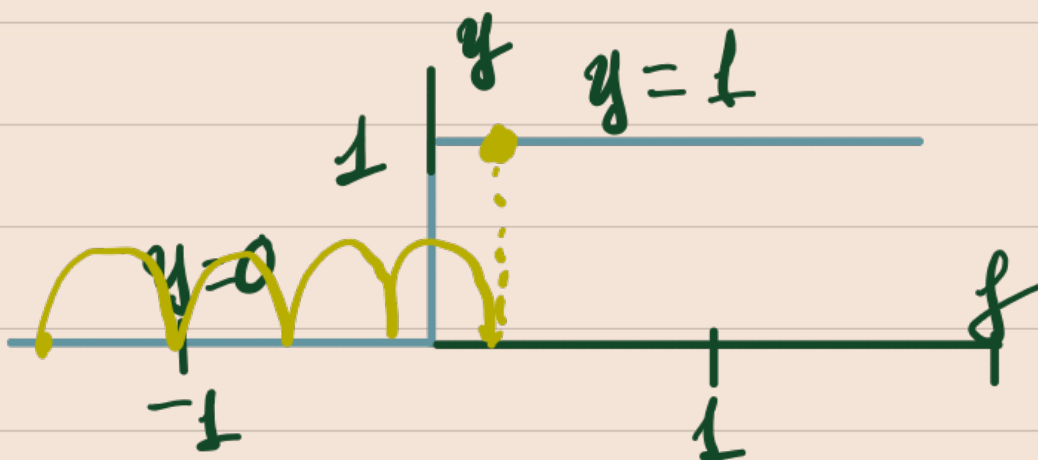
$\Rightarrow$  le coefficient  $w$  continuera d'augmenter grâce à ce formule

Enfin:

$w = w + 0$  : des paramètres arrêteront d'évoluer.

$$w_1 = w_1$$

$$w_2 = w_2$$



Le Perceptron est un modèle  
linéaire  $f = w_1 x_1 + w_2 x_2 + b$

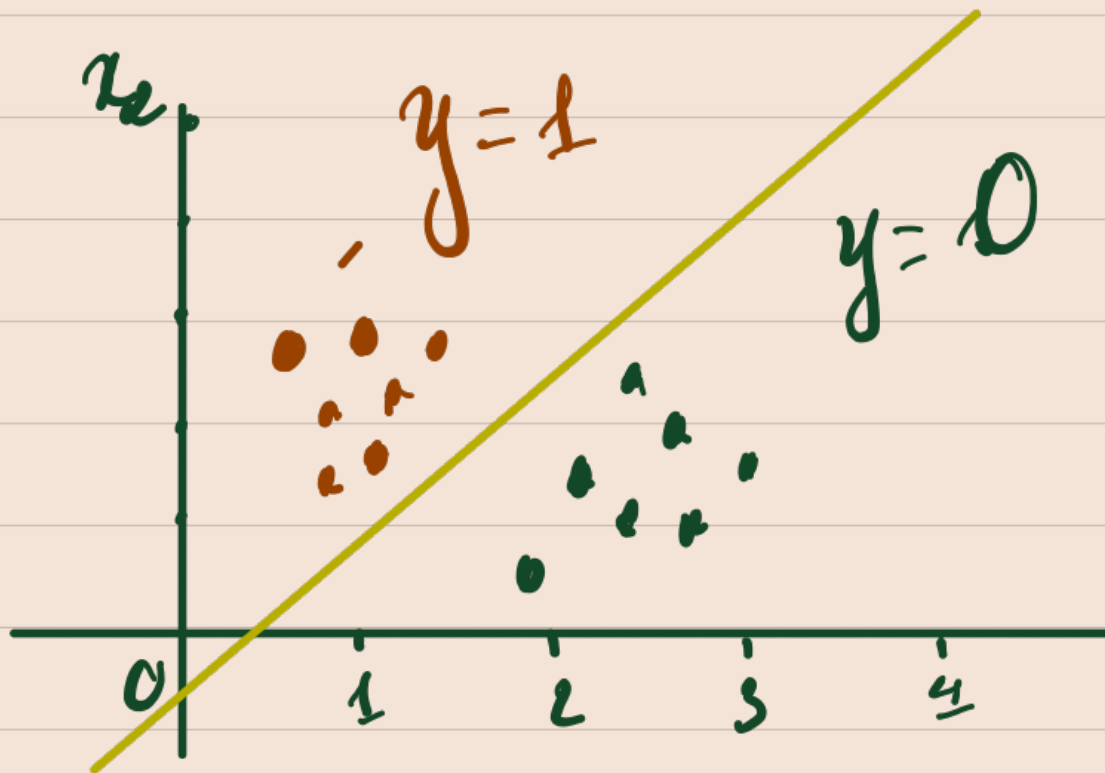
$$w_1 = -0,38$$

$$w_2 = 0,39$$

$$b = 0,1$$



position modifiée  
à l'aide d'un  
petit paramètre  
complémentaire  
biais

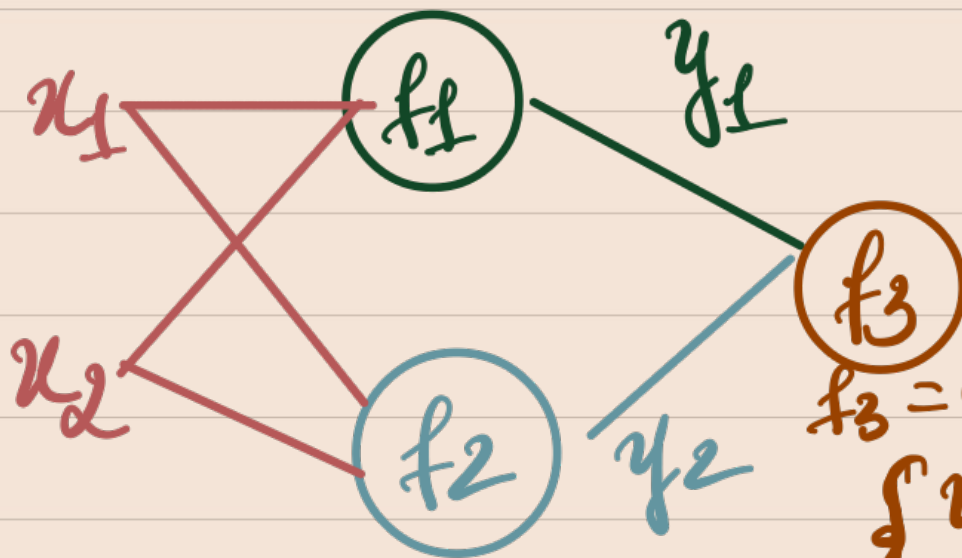


$$\begin{cases} y=1 & \text{si } f \geq 0 \\ y=0 & \text{sinon} \end{cases}$$

⊛ Seul ennui : une grande partie  
des phénomènes de notre univers  
ne sont pas des phénomènes linéaires.

$$f_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

$$\begin{cases} y_1 = 1 & \text{si } f_1 \geq 0 \\ y_1 = 0 & \text{sinon} \end{cases}$$



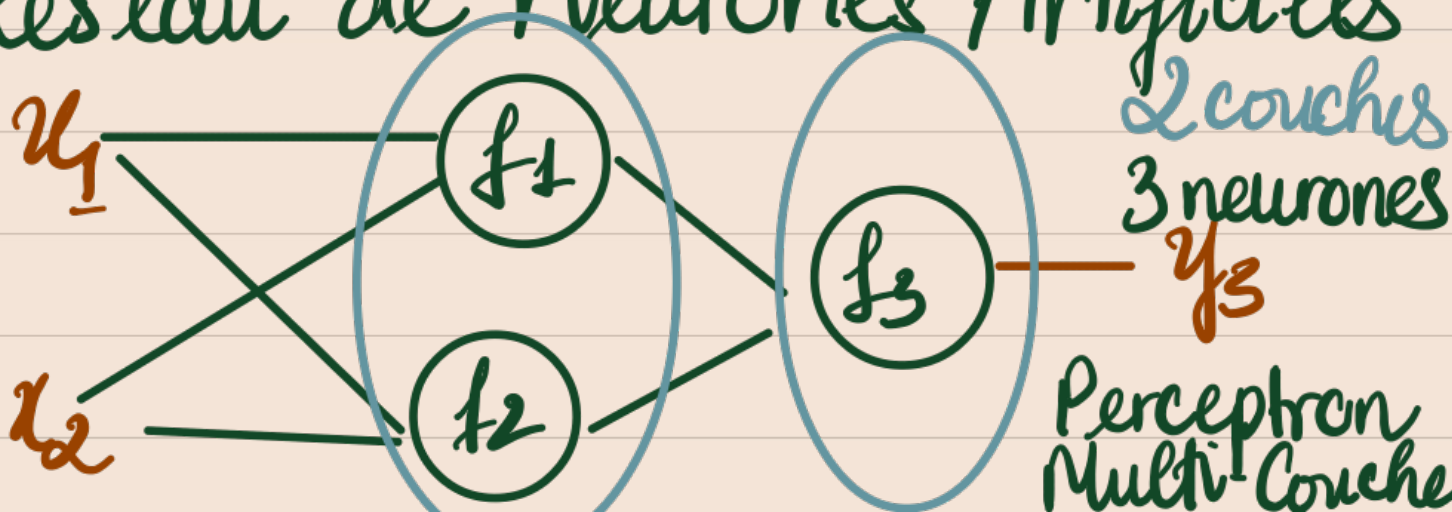
$$f_3 = w_{31}y_1 + w_{32}y_2 + b_3$$

$$\begin{cases} y_3 = 1 & \text{si } f_3 \geq 0 \\ y_3 = 0 & \text{sinon} \end{cases}$$

$$f_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

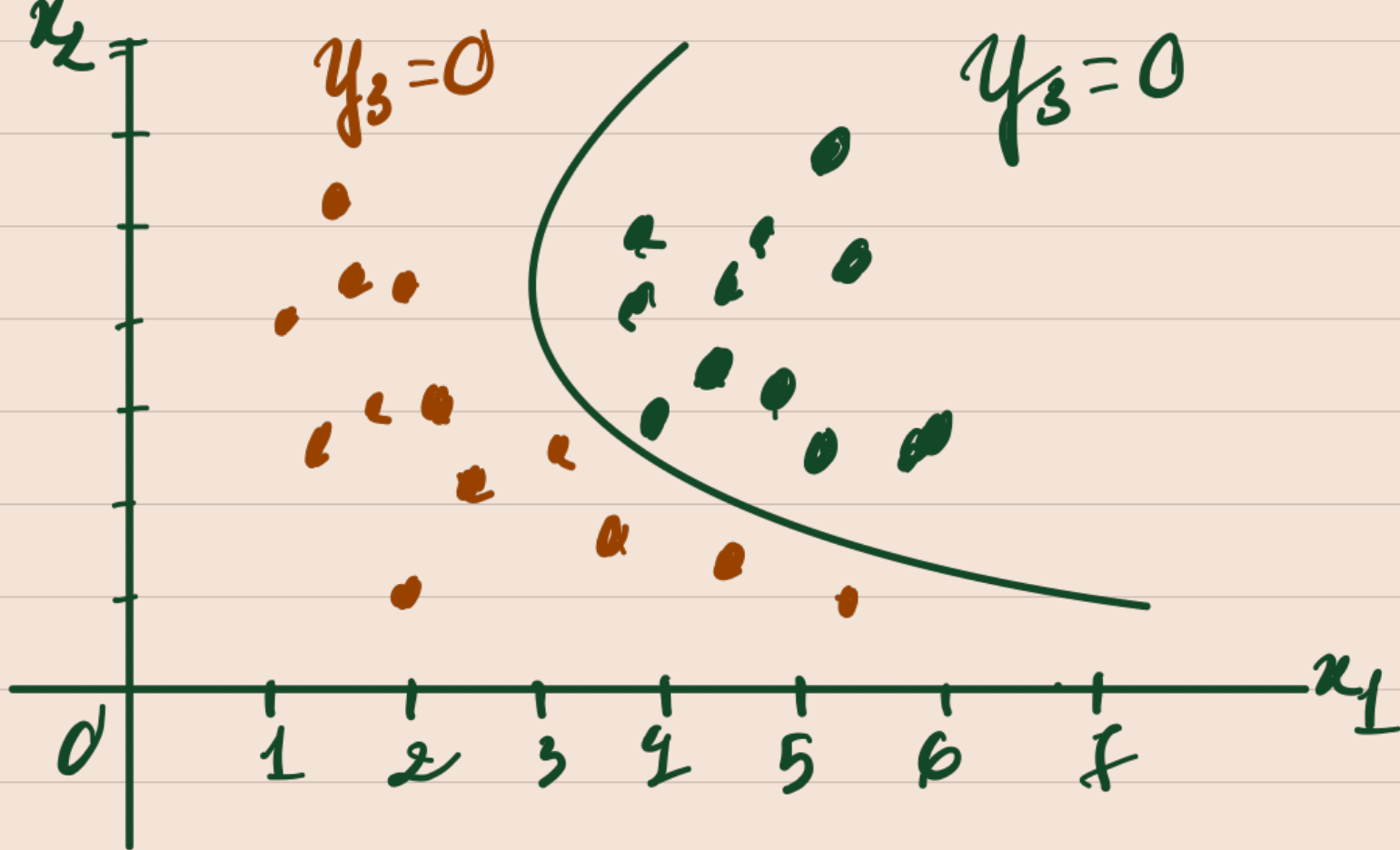
$$\begin{cases} y_2 = 1 & \text{si } f_2 \geq 0 \\ y_2 = 0 & \text{sinon} \end{cases}$$

Réseau de Neurones Artificiels



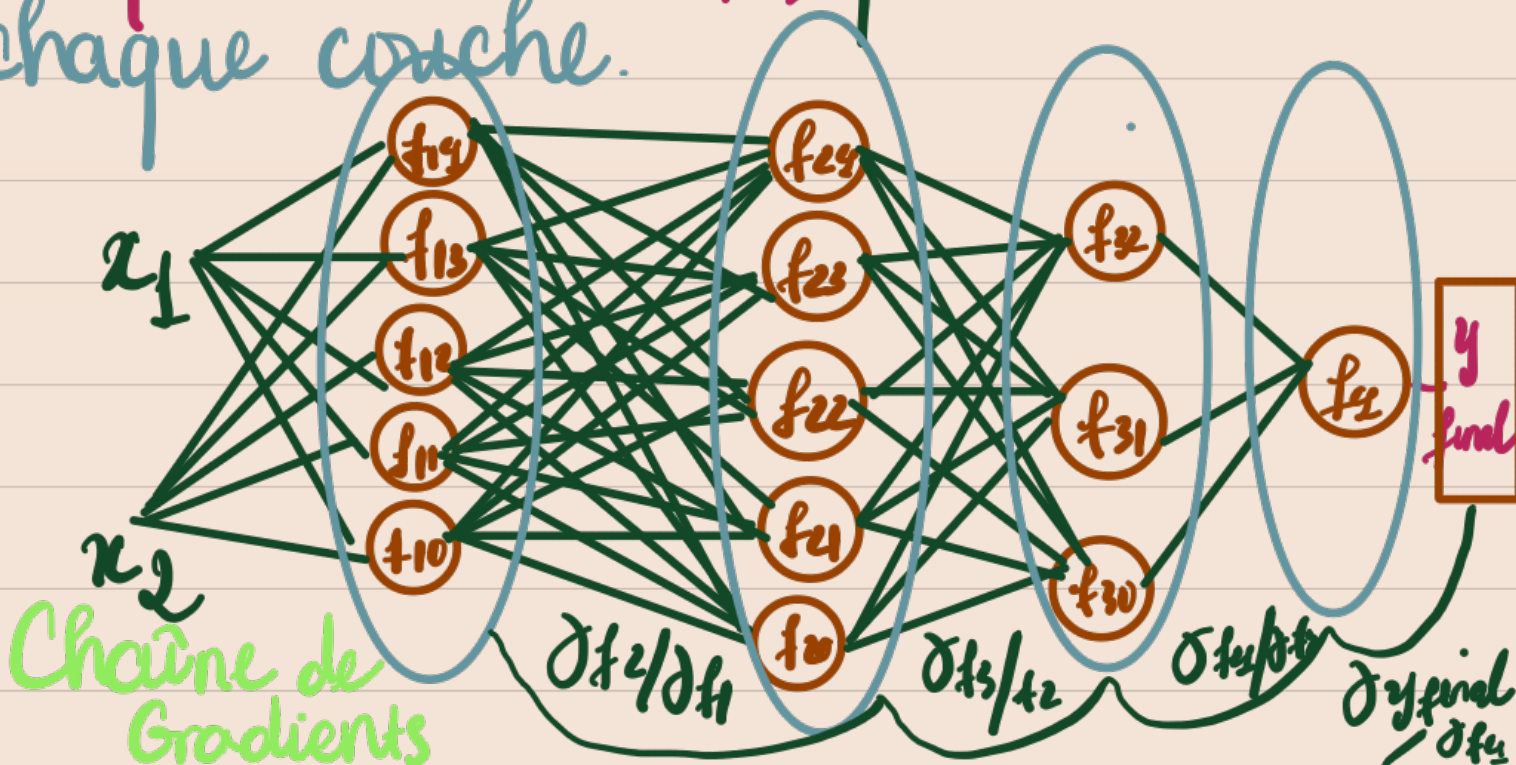
2 couches  
3 neurones

Perceptron  
Multi-Couche



## Back - Propagation

Consiste à déterminer comment la sortie du réseau varie en fonction des paramètres ( $W, b$ ) présents dans chaque couche.



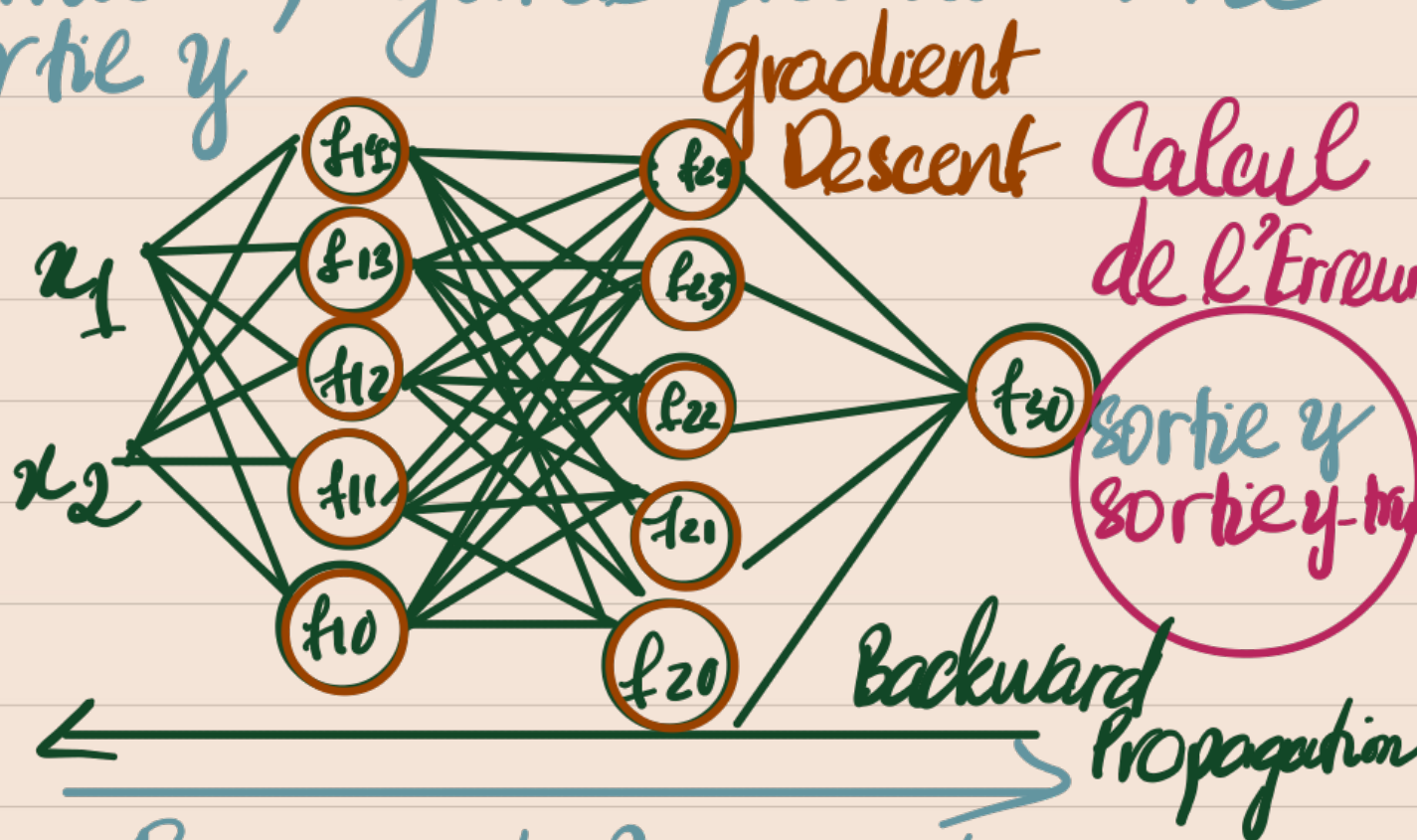
Grâce aux **gradients**, on peut alors mettre à jour les **paramètres** ( $W, b$ ) de chaque couche de telle sorte à ce qu'ils **minimisent** l'erreur entre la **sortie** du modèle & la **réponse attendue**.



$$W = W - \alpha \frac{\partial \text{Erreur}}{\partial W}$$

En résumé: Développer des Réseaux de Neurones Artificiels

1. Forward Propagation : première étape est circuler les données première couche jusqu'à la dernière, afin de produire une sortie  $y$



Forward Propagation

2. Cost Function : deuxième étape est calculer l'erreur entre cette sortie et la sortie de référence  $y_{true}$  que l'on désire avoir

3. Backward Propagation : partir de la dernière et remonter jusqu'à la toute première

4. Gradient Descent : corriger chaque paramètre du modèle grâce à l'algo de la descente de gradient, avant de se-boucler vers la première étage, celle de la Forward Propagation pour recommencer un cycle d'entraînement.

\* Note → Deep learning nécessite bcp de données  
→ Une grande puissance de calcul est nécessaire

⚠ Attention : Les réseaux de neurones ne fonctionnent pas comme notre cerveau.

- 1 Forward Propagation
- 2 Cost Function
- 3 Backward Propagation
- 4 Gradient Descent

Re - boucler